
Software Architecture, Agile and XP

Architecting Software Systems

Software Architecture, Agile and XP

Design decisions, iterative delivery and disciplined programming practices

- Advanced Programming Techniques
- Architecture views · quality attributes · agile principles · XP practices
- With Java examples, CI/CD, spec-driven engineering and AI-assisted development

architecture

agile

XP

TDD

CI

ADRs

AI

2026

Part I

Software architecture

- Architecture is about the important decisions that are hard to change later.
- A good architecture makes evolution safer, not just the first implementation prettier.

What Is Software Architecture?

The structure of the system and the reasoning behind it

- Main components, responsibilities and relationships
- Deployment decisions: where software runs and how parts communicate
- Data ownership, integration points, protocols and dependencies
- Quality attributes: performance, security, maintainability, scalability
- Trade-offs and constraints that explain why the system is built that way

Architecture is not only boxes and arrows. It is the set of decisions that shape future change.

Architecture vs Design vs Code

Different levels of decision

Architecture

System boundaries, components, deployment, integration, quality attributes

Design

Classes, interfaces, patterns, object collaboration, module responsibilities

Code

Methods, statements, tests, naming, local implementation details

The boundary is not perfect. A small code decision can become architectural when many modules depend on it.

Architecture Drivers

Architecture should respond to forces, not fashion

Functional requirements

What the system must do

Quality attributes

Performance, scalability, reliability, security, modifiability

Constraints

Legacy systems, team skills, budget, regulations, cloud provider

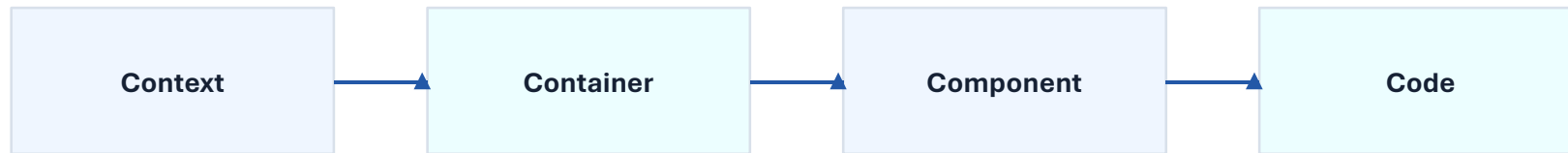
Change scenarios

What will probably change in the future

A microservice architecture is not good by itself. It is good only if it fits the drivers.

C4 Model: Four Architecture Views

Explain architecture at different zoom levels

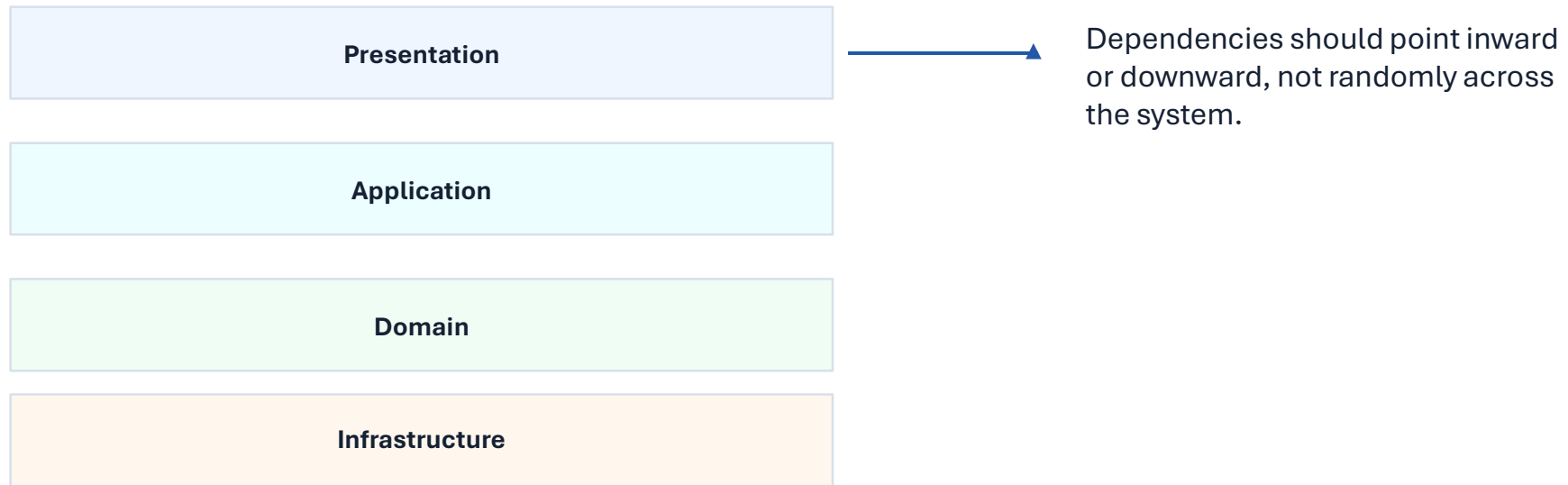


Zoom from system purpose to implementation detail

- Context: the system and its external users/systems
- Container: applications, databases, services and runtimes
- Component: modules inside a container
- Code: classes and implementation details when needed

Layered Architecture

A classic organization for business applications



- Presentation layer handles UI/API interaction
- Application layer coordinates use cases
- Domain layer contains business concepts and rules
- Infrastructure layer talks to databases, files and external services

Layered Java Example

Keep business logic away from infrastructure details

```
// Domain
public record Order(String id, List<OrderLine> lines) {}

// Application service
public final class PlaceOrderUseCase {
    private final OrderRepository repository;

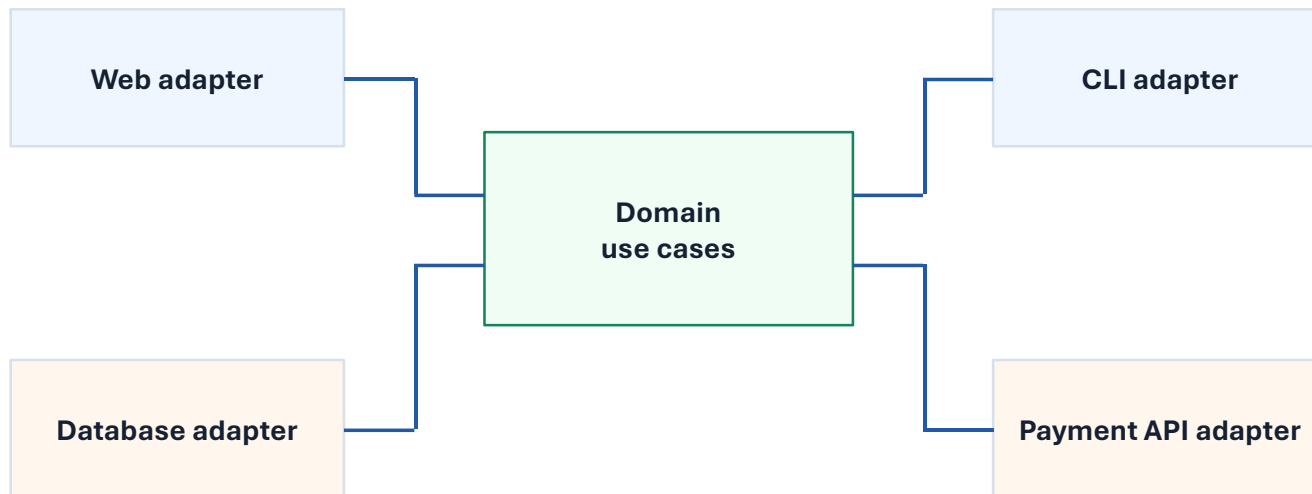
    public PlaceOrderUseCase(OrderRepository repository) {
        this.repository = repository;
    }

    public void place(Order order) {
        repository.save(order);
    }
}
```

- The use case depends on an interface
- The database implementation can change later
- The domain model is not tied to a web framework

Hexagonal Architecture

Ports and adapters isolate the domain



- The domain is in the center
- Ports define what the domain needs or offers
- Adapters connect web, database, CLI, messaging or external APIs
- Testing becomes easier because adapters can be replaced

Ports and Adapters in Java

Interfaces express architectural boundaries

```
public interface PaymentPort {
    PaymentResult charge(Order order);
}

public final class CheckoutService {
    private final PaymentPort payment;

    public CheckoutService(PaymentPort payment) {
        this.payment = payment;
    }

    public Receipt checkout(Order order) {
        PaymentResult result = payment.charge(order);
        return Receipt.from(order, result);
    }
}
```

- The core service does not know Stripe, PayPal or test doubles
- The port protects the use case from external technology changes
- This is architecture expressed as ordinary Java interfaces

Monolith vs Modular Monolith vs Microservices

Three different ways of managing boundaries



- A monolith is simple to deploy but can become tangled
- A modular monolith keeps one deployment but separates modules clearly
- Microservices add independent deployment and data ownership, but also distributed complexity
- For teaching, a modular monolith is often the best architecture to understand first

Microservices: Benefits and Costs

A distributed architecture is also a distributed failure model

Benefits

Independent deployment, team autonomy, scalability by service, technology flexibility

Costs

Network failures, observability, distributed data, versioning, testing, operations

Warning

If module boundaries are unclear in code, microservices will not magically fix them

Good question

Can this boundary own its data, tests, deployment and incidents?

Quality Attributes

Architecture is mostly about non-functional requirements

Performance

How fast must the system respond?

Scalability

How does the system behave with more users, data or requests?

Availability

How much downtime is acceptable?

Security

Who can access what, and how is data protected?

Maintainability

How easy is it to modify the system safely?

Architecture Trade-offs

There is no perfect architecture

Decision: split reporting into a separate service

Positive:

- + independent scaling
- + separate release cycle

Negative:

- network calls
- data synchronization
- more deployment complexity

Consequence:

add API contract tests and observability

- Every architectural decision has consequences
- Trade-offs should be explicit
- Students should learn to justify decisions, not memorize fashionable patterns

Architecture Decision Records

A lightweight way to document important decisions

```
# ADR 003: Use a modular monolith

## Context
The team is small and the domain boundaries are still evolving.

## Decision
Use one deployable application with explicit modules.

## Consequences
Simpler deployment, but module boundaries must be checked in code review.
```

- ADRs explain why a decision was taken
- They help future developers understand trade-offs
- They are very useful when AI tools modify the codebase

Part II

Agile programming

- Agile is not a ceremony checklist.
- Agile is a way to reduce uncertainty by delivering, learning and adapting.

Why Agile?

Software requirements are rarely stable

- Users discover what they need after seeing working software
- Technology, market and organizational constraints change
- Large up-front plans often hide risk until late
- Agile reduces risk by using short feedback loops
- The goal is controlled adaptation, not improvisation

Agile is a response to uncertainty.

Agile Manifesto: Four Values

The classical starting point

Individuals and interactions	over processes and tools
-------------------------------------	--------------------------

Working software	over comprehensive documentation
-------------------------	----------------------------------

Customer collaboration	over contract negotiation
-------------------------------	---------------------------

Responding to change	over following a plan
-----------------------------	-----------------------

The items on the right still have value. Agile says the items on the left matter more.

Agile Is Not No Documentation

Modern agile teams document differently

- Use lightweight artifacts: user stories, acceptance criteria, ADRs, README files
- Document decisions that are expensive to rediscover
- Keep documentation close to code and tests
- Prefer living documentation generated from tests, APIs or specs
- Spec-driven engineering can make agile documentation precise without becoming heavy

User Stories and Acceptance Criteria

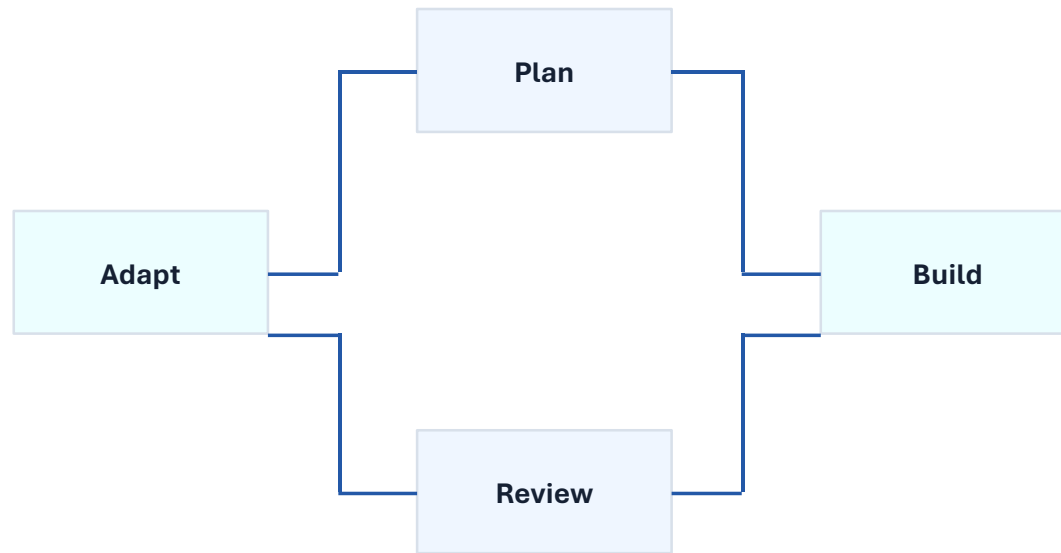
Describe value and observable behaviour

```
User story:  
As a teacher,  
I want to export grades to CSV,  
so that I can upload them to Moodle.  
  
Acceptance criteria:  
- Given a course with grades  
- When I export as CSV  
- Then the file contains one row per student  
- And missing grades appear as empty cells
```

- Stories describe user value
- Acceptance criteria define testable behaviour
- Good stories are small enough to implement and review

Agile Feedback Loop

Build small, learn fast, adapt safely



- Plan a small increment
- Implement with tests
- Review with users or stakeholders
- Adapt the backlog and architecture

Short feedback loops reduce uncertainty

Scrum and Kanban in One Slide

Two common agile management styles

Scrum

Sprints, sprint planning, daily scrum, review, retrospective, product backlog

Kanban

Continuous flow, visual board, work-in-progress limits, cycle time

Both

Make work visible, limit chaos, improve with feedback

For TAP

The programming practices matter more than the ceremony names

Part III

Extreme Programming

- XP is agile with strong engineering discipline.
- It connects requirements, tests, design, refactoring and continuous integration.

What Is Extreme Programming?

Agile development centered on programming practices

- Created to deliver high-quality software under changing requirements
- Makes good practices explicit and frequent: testing, refactoring, integration, feedback
- Emphasizes communication, simplicity, feedback, courage and respect
- Still highly relevant because modern AI-assisted coding needs the same safety nets

XP is not just working fast. It is working with fast feedback.

XP Values

The cultural base of the practices

Communication

Developers, users and stakeholders share understanding

Simplicity

Build the simplest design that works now

Feedback

Tests, users and integration reveal reality quickly

Courage

Refactor, delete, improve and change direction when evidence demands it

Respect

Code quality and collaboration are team responsibilities

XP Core Practices

Engineering discipline as a routine

- Test-driven development
- Pair programming or collaborative programming
- Continuous integration
- Refactoring
- Simple design
- Collective code ownership
- Small releases
- Coding standards

TDD: Red, Green, Refactor

A design technique, not only a testing technique

```
// Red: write a failing test
@Test
void appliesVipDiscount() {
    assertEquals(90, pricing.priceFor(vipCustomer, item100));
}

// Green: make it pass
// Refactor: simplify the design while tests stay green
```

- Red: specify the next behaviour as a failing test
- Green: implement the simplest code that passes
- Refactor: improve design without changing behaviour
- The cycle keeps design and validation close together

Pair Programming

Two minds, one code stream

Driver

Writes code and thinks tactically

Navigator

Reviews, asks questions, thinks strategically

Benefits

Knowledge sharing, fewer defects, better design discussion

Modern variant

Human plus AI assistant, with the human acting as reviewer and architect

Continuous Integration

Integrate early, test automatically

```
name: Java CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
      - run: mvn test
```

- Every change should be integrated frequently
- Automated tests protect the team from silent regressions
- CI turns XP discipline into an executable workflow

Refactoring in XP

Improve the design continuously

```
// Before: one method does validation, pricing and persistence
placeOrder(request);

// After: responsibilities are visible
validator.validate(request);
Price price = pricingPolicy.calculate(request);
repository.save(order);
```

- Refactoring is not a separate late phase
- TDD provides the safety net for small design improvements
- Common refactorings: Extract Method, Move Method, Extract Class, Replace Conditional with
- **Polymorphism**
The architecture evolves through many small, tested changes

Simple Design

Build for today's known need, but keep change cheap

- Passes all tests
- Expresses intention clearly
- Contains no duplication
- Has the fewest elements needed now
- Avoids speculative generality: do not design for imaginary future requirements

Simple does not mean naive. Simple means easy to understand and change.

Collective Code Ownership

The team owns the whole codebase

- Any developer can improve any part of the code
- Requires tests, coding standards and shared design understanding
- Avoids knowledge silos and fragile ownership boundaries
- Works especially well with pull requests and continuous integration

Agile Architecture

Architecture should evolve, but not disappear

- Start with enough architecture to reduce the biggest risks
- Use prototypes or spikes for uncertain technology choices
- Keep architectural decisions visible through ADRs and diagrams
- Refactor architecture incrementally when new evidence appears
- Avoid both extremes: no architecture and big design up front

Spec-Driven Engineering and Agile

Specifications can make agile work more precise

```
change-add-csv-export/  
  proposal.md  
  design.md  
  tasks.md  
  specs/  
    report-export.md
```

Each task should map to tests, commits and review evidence.

- Specs clarify intent before implementation
- Tasks make work visible and reviewable
- Acceptance criteria connect requirements to tests
- Small specs fit agile increments better than long frozen documents

AI-Assisted Agile Development

AI changes speed, not responsibility

- AI can draft tests, code, refactorings, documentation and ADRs
- The team must still define intent, boundaries and acceptance criteria
- XP practices become more important: tests, CI, refactoring and review
- AI-generated changes should be treated like junior developer contributions: useful, but reviewed

AI makes disciplined engineering practices more necessary, not less.

Agentic Programming Workflow

From story to implementation through supervised automation

1. Write user story and acceptance criteria
2. Create a small technical spec
3. Ask the agent to implement one task
4. Run tests and static checks
5. Review the diff and architecture impact
6. Iterate or reject the change

- Agents can plan and execute, but they need constraints
- Small tasks reduce risk
- Tests and quality gates become the agent's feedback environment

Mini-Project

Architecture + XP + spec-driven implementation

Deliverables:

- story.md
- adr-001.md
- tests
- implementation
- short reflection

- Choose a small feature for the TAP project
- Write a user story and acceptance criteria
- Create one ADR for an architectural decision
- Implement using TDD or test-first thinking
- Use GitHub Actions to run the tests
- Reflect on which XP practices helped control the change

Key Takeaways

Architecture and agility are complementary

- Architecture is the set of important decisions that guide change
- Agile reduces uncertainty through short feedback loops
- XP adds engineering discipline: TDD, CI, refactoring, simple design and collaboration
- Spec-driven engineering gives AI and humans a shared source of intent
- Modern software teams need both architecture thinking and fast, validated iteration

The goal is not to choose between design and agility. The goal is to make change safe.